

2.04. Le rectangle

2.04 : le rectangle — plus large que haut. Un dessin de 16×16 posé au milieu de l'écran avec `sprite16()`, son nom dessous.

1. Ce qu'il faut comprendre

Un rectangle a **4 coins droits**, comme le carré, mais ses côtés ne sont pas tous égaux : les côtés **face à face** ont la même longueur. Ici il est **plus large que haut** : 14 pixels de large, 8 de haut. Un carré est un rectangle dont les quatre côtés sont égaux.

Il s'écrit comme le carré, avec moins de rangées : une rangée de `#` en haut, une en bas, et entre les deux des rangées qui commencent et finissent par `#`. Les rangées vides (`.`) au-dessus et en dessous le centrent dans les 16×16 .

Le programme est celui du 2.01 : seuls changent le fichier du dessin (`perso_RECTANGLE.cpp`), le nom `RECTANGLE` dans `sprite16()`, et le mot écrit dessous. `#` est le plus sombre, `+` moyen, `-` clair, `.` le transparent.

Essaie : compte les rangées du rectangle, puis ajoute-en deux pour qu'il fasse 10 pixels de haut (en prenant deux rangées vides).

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <sprite16>` → [35.2. La fonction sprite16\(\) — un personnage de \$16 \times 16\$](#)
- `#include <Perso>` → [35.1. Perso — dessiner des personnages](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **602 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      57 o ← la console, à l'allumage — la mise en route de la console
main()         117 o ← démarrage — ta fonction
VBlank         8 o  ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage  28 o ← image() — attend l'image suivante
AttendreVBlank 16 o ← démarrage — attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte — attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() — écrit un texte, lettre par lettre
RangerLutins   11 o ← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert 15 o ← démarrage — installe la copie des lutins en mémoire rapide
CopierTuiles   103 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage — vide les deux cartes de l'écran
Tuiles         192 o ← CopierTuiles — 8 lettres sur 43, et 4 tuiles dessinées
DonneesTransfert 10 o ← InstallerTransfert — la petite routine qui copie les lutins
"RECTANGLE"    9 o  ← texte() — un texte gravé
Lettres gravées : ACEGLNRT
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste,
```

Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, LireManette, Notes

Fonctions du langage appelées :

```
sprite()   x 4    96 o sur place, dans main()
texte()    x 1    11 o sur place, dans main() + EcrireTexte (10 o)
image()    x 1     6 o sur place, dans main() + AttendreImage (28 o)
sprite16() x 1     0 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- 2.04 Le rectangle ----
// Le rectangle de 16 x 16 au milieu de l'écran, et son nom dessous.

#include <texte>                // texte() : écrire un mot à l'écran
#include <sprite16>             // sprite16() : poser un personnage de 16 x 16
#include <Perso>                 // Perso : un dessin de 16 x 16
#include "perso_RECTANGLE.cpp"  // le dessin, dans son fichier
```

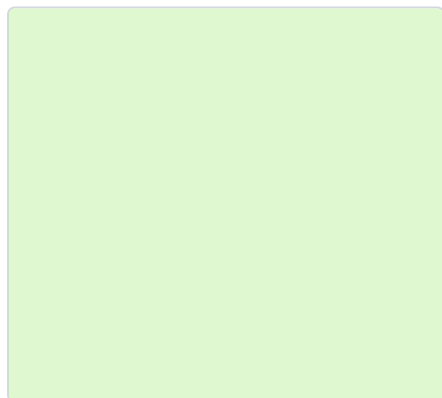
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    sprite16(0, 72, 56, RECTANGLE);    // les lutins 0 à 3, coin haut-gauche en (72, 56)
    texte(5, 10, "RECTANGLE");        // le nom, sous la forme

    while (true) {    // la boucle du jeu : rien ne bouge, on regarde
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Le rectangle au milieu de l'écran, et RECTANGLE écrit dessous.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ la forme est posée au milieu, en (72, 56) (72, 56)

✓ ses quatre lutins font un carré de 16

✓ son nom est écrit dessous : RECTANGLE

Ensuite : **2.05. Le losange** — série 2 : Les formes géométriques. · gameboy3 — écrire une cartouche Game Boy en C++